Making Submission `8e9c9a2` Compatible with Coherent Windowed Point Addition

Runtime-table ABI, quantum-circuit construction, and validation

ECDSA.Fail Supplemental Technical Note

9 August 2026

Abstract

Submission `8e9c9a2` implements mixed elliptic-curve point addition with a classically specified addend. A windowed Shor circuit instead supplies the addend coherently: a quantum address selects a row of a classical point table, the selected point is added to a quantum accumulator, and all address-dependent workspace must be removed. We modify the circuit to expose the quantum registers X, Y, J and $2 \cdot 2^w$ classical coordinate registers, condition QROM payload operations on those runtime table bits, derive $3x_J$ inside the circuit, handle the identity row coherently, and exactly replay each lookup after use. For $w = 16$, the resulting circuit uses $Q = 1{,}162$ logical qubits and $T = 1{,}684{,}156.770$ average executed Toffoli gates over 9,024 cases. It has no ancilla failures and an any-channel error rate of 0.2216%, arising from finite-support arithmetic approximations. Correlated four-call tests for every $w \in \{0, \ldots, 5\}$ exercise 6,144 additions and confirm cleanup after every call. The construction implements a reusable window-selected point-addition kernel on the same generic-affine domain as the source circuit; it is not a complete implementation of Shor's algorithm.

1 Target Interface and Runtime ABI

Let $R = (x_R, y_R)$ be the quantum accumulator, let J be a coherent w-qubit address, and let $A_J = (x_J, y_J) = \mathcal{T}[J]$ be the selected point. The required map is

$$U_{\text{win}} : \quad |J\rangle\,|R\rangle\,|0\rangle_W \longmapsto |J\rangle\,|R + A_J\rangle\,|0\rangle_W\,, \tag{1}$$

where W contains the QROM payload, routing qubits, arithmetic scratch, and cleanup state. The address must remain coherent and unchanged, all arithmetic must depend coherently on the selected row, and no row-dependent payload, phase, or ancilla may survive the call.

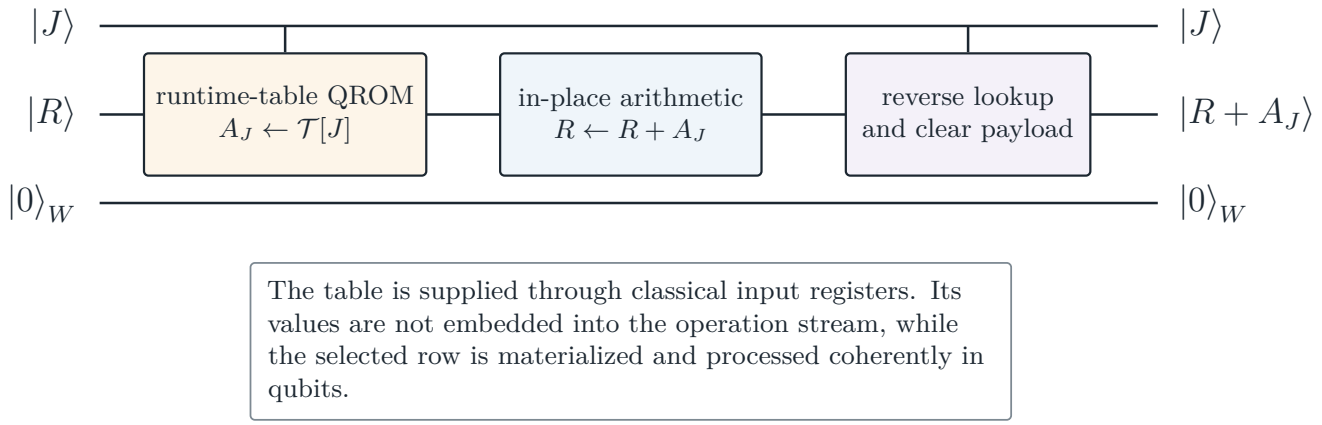

Figure 1: Runtime-table window interface implemented by the modified circuit.

The emitted register ABI is

$$(X, Y, J, \mathcal{T}_x[0], \mathcal{T}_y[0], \ldots, \mathcal{T}_x[2^w - 1], \mathcal{T}_y[2^w - 1]), \tag{2}$$

where X and Y are 256-qubit coordinate registers, J is a w-qubit quantum register, and every table coordinate is a 256-bit classical input register. Thus, a $w = 16$ instance exposes $2 \cdot 2^{16}$ coordinate registers. The operation stream refers to their bits through classical conditions; replacing one shifted G- or public-point table by another changes the input data rather than the circuit topology. The supplied evaluator populates these registers with the reference table $\mathcal{T}[j] = [j]G$, but the emitted circuit is not restricted to that table.

2 Circuit-Level Modifications

2.1 Stage-local materialization

The six-stage arithmetic order of `8e9c9a2` is retained. Only the boundaries at which the addend is consumed are changed. Stages 1 and 6 jointly load the two selected coordinates through one address traversal; Stage 3 loads only x_J. Every payload is removed before the dialog-GCD peaks.

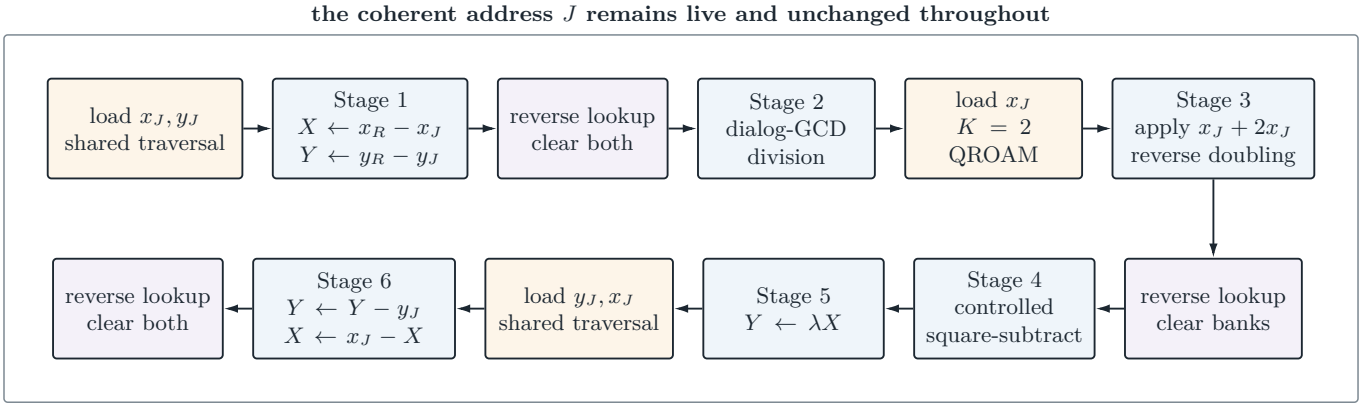

Figure 2: Stage-local lookup/use/replay schedule. The selected point is never retained through the arithmetic peak.

Stage 3 does not require a derived $3x$ table. It loads x_J into a 257-qubit temporary register, first accumulates x_J into X, reversibly doubles the temporary value modulo p, accumulates $2x_J$, reverses the doubling, and unloads the original x_J. Therefore, the update is $X \leftarrow X + 3x_J \pmod p$, while the ABI contains only the two source-coordinate columns. Stage 6 likewise derives reverse subtraction from the loaded x_J: for a nonidentity row it applies modular negation to X and then adds x_J, giving $x_J - X \pmod p$. No embedded $x_J + 1$ column is required.

2.2 Identity row and generic-affine domain

Row $J = 0$ represents the elliptic-curve identity $\mathcal{O}$. The circuit computes a coherent predicate $b = [J \neq 0]$, uses b to control the square-subtract in Stage 4 and modular negation in Stage 6, and uncomputes b at the end. When $J = 0$, the QROM payloads are zero, so Stages 1–3 leave (X, Y) at $(x_R, y_R x_R^{-1})$ after division; Stage 4 is disabled; Stage 5 reverses the division map and restores $Y = y_R$; and Stage 6 leaves $X = x_R$. Hence the complete branch returns $R + \mathcal{O} = R$ without measuring J.

The exceptional affine cases $R = A_J$ and $R = -A_J$ remain outside the circuit domain. They make the denominator $x_R - x_J$ vanish and require point doubling or an explicit identity output. This is the same generic-affine restriction used by the source circuit and Schrottenloher's point-addition construction; it is distinct from the table's identity row, which the modified circuit now handles.

2.3 Low-peak arithmetic

The QROM payload is live only near its consumer, but it still competes locally with carry ancillas. The modification suppresses carry venting at loaded-data sites and retains only a 64-bit upper comparison during its low-peak modular additions. This support-calibrated approximation exchanges additional Toffoli work and a small correctness risk for fewer simultaneously live carry qubits. The global maximum remains the inverse-application fold after all QROM payloads have been cleared.

3 Runtime QROM and Exact Replay

3.1 Why an XOR lookup loads a table word

For one table column $c_j \in \{0,1\}^{256}$, QROM implements

$$L_c : \quad |j\rangle\,|z\rangle \longmapsto |j\rangle\,|z \oplus c_j\rangle\,. \tag{3}$$

If the payload starts in $|0\rangle$, then $0 \oplus c_j = c_j$, so $L_c\,|j\rangle\,|0\rangle = |j\rangle\,|c_j\rangle$. Operationally, a reversible address traversal constructs the predicate selecting row j and applies a classically conditioned CNOT to each payload position whose runtime table bit is one. By linearity,

$$\sum_j \alpha_j\,|j\rangle\,|0\rangle \longmapsto \sum_j \alpha_j\,|j\rangle\,|c_j\rangle\,, \tag{4}$$

so the payload becomes correlated with the coherent address. Here c_j is one coordinate column, such as x_j or y_j, not the complete point A_j.

3.2 $K = 2$ QROAM and payload width

The single-column Stage 3 lookup uses two-row blocking. Write $j = 2h + \ell$, where h contains the high $w - 1$ address bits and ℓ is the least significant bit. The high-address traversal loads adjacent rows into two 256-qubit banks, and controlled swaps select the requested row:

$$|h,\ell\rangle\,|0\rangle_{B_0}\,|0\rangle_{B_1} \longmapsto |h,\ell\rangle\,|c_{2h}\rangle_{B_0}\,|c_{2h+1}\rangle_{B_1} \longmapsto |h,\ell\rangle\,|c_{2h+\ell}\rangle_{B_0}\,|c_{2h+1-\ell}\rangle_{B_1}\,. \tag{5}$$

Because $j = 2h + \ell$, $c_{2h+\ell} = c_j$ is precisely the selected word. The notation does not mean that $c_{2h+\ell}$ is the complete table point: in Stage 3 it is the x-coordinate word x_j.

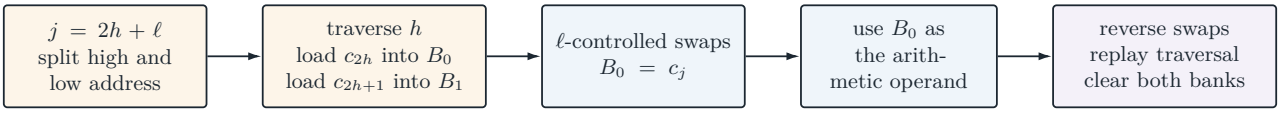

Figure 3: $K = 2$ QROAM used for the Stage 3 x_J episode.

The construction does not store a 512-bit point in the 16 address qubits. Stages 1 and 6 temporarily allocate two 256-qubit payloads, and Stage 3 temporarily allocates the selected and adjacent 256-qubit banks shown in (5). The width saving is temporal: these banks are cleared before the global dialog-GCD peak instead of remaining resident throughout the point addition. Consequently, their local 512-qubit storage increases the measured peak by only 11 qubits relative to the mixed-addition circuit.

3.3 Exact replay unload

After the arithmetic consumer preserves a loaded payload, applying the same XOR lookup again gives

$$L_c^2 : \quad |j\rangle\,|c_j\rangle \longmapsto |j\rangle\,|c_j \oplus c_j\rangle = |j\rangle\,|0\rangle\,. \tag{6}$$

The reported circuit therefore reverses the $K = 2$ swaps when applicable and replays the same runtime-table traversal. This exactly removes the payload without measuring it. The traversal's unary-routing AND ancillas are removed using measurement-based uncomputation with conditional phase correction, yielding a clean coherent QROM boundary with no address-dependent residual phase. A measurement-based *payload* unload remains implemented as an optional experiment, but the final resource and correctness results use payload replay.

4 Circuit-to-Source Correspondence

These changes establish window compatibility because J is a genuine quantum register, every table-dependent payload operation is controlled by the externally supplied table bits, arithmetic consumes qubit payloads, and exact replay removes all which-row information outside J. The resulting circuit topology can therefore be reused with the shifted tables $\mathcal{T}_G^{(i)}[j] = [j2^{iw}]G$ or $\mathcal{T}_P^{(i)}[j] = [j2^{iw}]P$ required by a complete windowed scalar multiplication.

Episode	Load Toffoli	Replay Toffoli	Total	Runtime columns
Coordinate differences	65,534	65,534	131,068	x_J, y_J
Apply $3x_J$ $(K = 2)$	33,022	33,022	66,044	x_J
Output recovery	65,534	65,534	131,068	y_J, x_J
Total	164,090	164,090	328,180	six traversals

Table 1: Instrumented nonlinear QROM cost for $w = 16$. The complete stream also contains 167,772,160 classically conditioned table-payload CNOTs across loading and replay.

Circuit role	Principal source entry points	Effect
Runtime ABI	`trailmix_ludicrous/mod.rs`: `build_trailmix_ludicrous_ops`	Declares X, Y, J and the $2 \cdot 2^w$ classical coordinate registers; no table values are written into the emitted circuit.
Runtime payload controls	`qrom.rs`: `xor_payload`, `load_word`, `load_two_words`; `point_add/mod.rs`: `cx_if`	Emits CNOT operations whose classical condition is the corresponding runtime table bit.
Stages 1 and 6	`ec_add.rs`: `qrom_coord_differences`, `qrom_coord_recover`	Loads two coordinate columns through a shared traversal, applies the coordinate maps, and immediately replays the lookup.
Runtime $3x_J$	`ec_add.rs`: `qrom_coord_add3x`	Loads only x_J, applies $x_J + 2x_J$, reverses the modular doubling, and unloads x_J.
Identity handling	`ec_add.rs`: `toggle_nonzero_address`; `square.rs`: controlled square-subtract	Constructs $[J \neq 0]$, controls the two operations that distinguish the identity branch, and uncomputes the predicate.
Exact unload	`qrom.rs`: `unload_word`, `unload_two_words`, `emit_qroam2_replay_unload`	Reverses swaps and replays the XOR lookup to clear payload and routing qubits exactly.
Peak-width control	`arith.rs`: low-peak modular add/subtract variants	Avoids simultaneous payload and wide carry-vent allocations and uses a 64-bit upper comparison at the three QROM-use sites.
Validation only	`eval_circuit.rs`: window case generation, runtime table initialization, parallel checkpointing, and correlated sequence runner	Supplies arbitrary table values and verifies coordinates, address, incremental phase, and non-interface ancillas. These changes do not alter the emitted circuit.

Table 2: Correspondence between circuit-level changes and implementation entry points.

5 Resource Measurements

The fully emitted $w = 16$ circuit contains 213,387,745 operations. Its measured peak remains the inverse-application fold, where all QROM payloads have already been released. Table 3 compares the result with the reported `8e9c9a2` operating point. The two executed-Toffoli values are measured on their respective deterministic evaluator supports, so the table characterizes resource overhead rather than a paired error experiment.

Metric	Original `8e9c9a2`	Runtime-table $w = 16$	Change
Peak logical width Q	1,151	1,162	+11 (+0.96%)
Average executed Toffoli T	1,299,453	1,684,156.770	+384, 703.770 (+29.61%)
$Q \times T$	1,495,670,403	1,956,990,167	+30.84%
Static emitted operations	9,133,449	213,387,745	23.36×

Table 3: Resource overhead of coherent runtime-table selection. Table-payload CNOTs dominate the static stream and are not counted as Toffoli gates.

The final circuit uses one more qubit and more Toffoli work than the earlier fixed-table, measurement-unload prototype. That increase is the cost of closing the principal interface and cleanup gaps: table data are now runtime inputs, and every payload is removed by exact replay. The persistent $w = 16$ address itself does not imply a 16-qubit increase in Q, because the original and modified circuits have different concurrent scratch allocations at their maxima.

6 Validation

6.1 Full 16-bit evaluation

The final 16-bit circuit was evaluated on 9,024 deterministic pseudorandom cases. Each case supplies the complete runtime table $\mathcal{T}[j] = [j]G$, samples a target accumulator and coherent address, and classically computes the expected generic-affine sum. Cases with nonzero J and $x_R = x_J$ are excluded because they fall outside the declared generic-affine domain; $J = 0$ is retained.

Outcome	Count
Classical mismatches	14 / 9,024 (0.1551%)
Phase-failure shots	10 / 9,024 (0.1108%)
Ancilla-failure shots	0 / 9,024 (0%)
Shots failing any channel	20 / 9,024 (0.2216%)
Identity selections	1 / 9,024 (0.0111%)

Table 4: Validation of the final $w = 16$ runtime-table circuit. Failure categories overlap.

The identity case returned the input coordinates, preserved $J = 0$, and had no classical, phase, or ancilla failure. Exact payload replay produced no QROM cleanup failure on any shot. The residual classical and phase failures are consistent with the fixed-length dialog-GCD schedule, inherited truncated arithmetic, and the window adapter's 64-bit upper comparison; the present experiment does not supply a coherent failure flag for these approximations.

The evaluator records one durable row per input and one commit record per 64-shot batch. On restart, it truncates any uncommitted input tail, reconstructs aggregate counts, and evaluates only unfinished batches. This checkpoint mechanism produced the complete 9,024-case ledger used above.

6.2 Correlated multi-call tests for $w = 0, \ldots, 5$

To test composition rather than isolated calls, the evaluator retains one accumulator, supplies a new coherent address, applies the same emitted kernel four times, and checks the intermediate accumulator, preserved address, phase increment, and every non-interface qubit after each call. For each $w \in \{0, \ldots, 5\}$, 256 four-call sequences were evaluated, giving 1,024 additions per width and 6,144 calls in total.

w	Identity selections	Any-failure sequences	Ancilla failures	Per-call average T
0	1,024 / 1,024	0 / 256	0	1,312,343.476
1	473 / 1,024	2 / 256	0	1,321,065.429
2	253 / 1,024	0 / 256	0	1,322,636.318
3	125 / 1,024	4 / 256	0	1,324,426.755
4	63 / 1,024	0 / 256	0	1,326,497.064
5	30 / 1,024	1 / 256	0	1,328,558.680

Table 5: Correlated four-call tests. The sparse classical/phase failures reflect the inherited approximate arithmetic; no sequence leaves QROM or arithmetic ancillas live.

The $w = 0$ circuit consists entirely of identity selections and is exact on this corpus. The $w = 1$ emitter and unload path also complete normally, closing the earlier base-case crash. Most importantly, the previous failure rate proportional to 2^{-w} disappears: row zero is no longer treated as the affine point $(0, 0)$.

7 Reproduction

The implementation is maintained on branch `codex/windowed-8e9c9a2` of `jieyilong/ecdsafail-challenge`. From its worktree:

```
cargo build --release --locked --offline \
  --bin build_circuit --bin eval_circuit

WINDOWED_MODE=1 WINDOW_BITS=16 SINGLE_CCX_FANOUT_DISABLE=1 \
  ./target/release/build_circuit

WINDOWED_MODE=1 WINDOW_BITS=16 WINDOWED_TESTS=9024 \
  WINDOWED_MAX_ERROR_RATE=1 EVAL_THREADS=8 \
  EVAL_CHECKPOINT_DIR=<checkpoint-dir> \
  ./target/release/eval_circuit \
  --note runtime-table-k16-replay-unload
```

Set `WINDOWED_SEQUENCE_CALLS=4` to select the correlated test mode; repeat emission and evaluation with `WINDOW_BITS=0`, ..., `5` to reproduce Table 5.

8 Limitations

- **Generic-affine exceptional cases.** The identity table row is supported, but $R = A_J$ and $R = -A_J$ remain outside the circuit domain, as in the underlying generic-affine construction.
- **Approximate arithmetic.** The inherited fixed-length dialog-GCD, finite-width arithmetic, and window-specific 64-bit upper comparison produce sparse classical and phase errors. The modified circuit does not convert these events into a coherent failure flag.
- **Finite-support evidence.** The experiments establish behavior only on the stated corpora; they are not an all-input proof or a coherent full-attack error estimate.
- **Kernel rather than full Shor.** The branch implements a reusable runtime-table point-addition call. A complete attack must instantiate all shifted G and P tables, schedule all windows, and validate the complete double-scalar multiplication and Fourier/postprocessing layers.
- **Accounting convention.** Resource counts follow the challenge model. A fault-tolerant architecture may assign different costs to classical table storage, Clifford gates, measurements, and feed-forward.

9 Conclusion

The revised construction closes the principal gaps of the fixed-table prototype. It exposes the complete classical point-table ABI, conditions QROM on runtime table bits, derives temporary values within the circuit, handles the identity row, supports $w = 0$ and $w = 1$, and exactly clears every payload by replay. The resulting 16-bit kernel stays within the requested width bound at $Q = 1{,}162$, with $T = 1{,}684{,}156.770$ and a measured any-channel error rate of 0.2216%. Its remaining limitations are finite-support arithmetic approximations, the standard generic-affine exceptional cases, and the absence of full-Shor integration rather than defects in the lookup/use/unload interface.